0678. Valid Parenthesis String

dium | Category: Greedy | LeetCode: https://leetcode.com/problems/valid-paren

Problem

- S has '(', ')' and '', where '' can be '(', ')' or empty. Is s valid?

Intuition

- Track the range [lo, hi] of possible open-bracket counts.
- '(' : lo++, hi++. ')' : lo--, hi--. '' : lo-- (treat as ')'), hi++ (treat as '(').
- If hi < 0 there are too many ')' - invalid. Clamp lo at 0. Valid iff lo == 0 at the end.

Brute Force

- Try all 3^k replacements of the stars: exponential. DP over (index, open) is O(n^2).

Optimal Approach

- Lo = hi = 0.
- Update per character as above; if hi < 0 return false; lo = max(lo, 0).
- Return lo == 0.

Worked Example

```
"())".

'(' lo=1 hi=1
'*' lo=0 hi=2
')' lo=0 (clamped from -1) hi=1
')' lo=0 (clamped) hi=0
end lo == 0 -> true
```

Java Solution

```java
public class ValidParenthesisString {
    public boolean checkValidString(String s) {
        int lo = 0, hi = 0;
        for (char c : s.toCharArray()) {
            if (c == '(') { lo++; hi++; }
            else if (c == ')') { lo--; hi--; }
            else { lo--; hi++; }
            if (hi < 0) return false;
            lo = Math.max(lo, 0);
        }
        return lo == 0;
    }
}
```

Complexity

- Time: O(n). Space: O(1).

Pitfalls

- Clamping lo at 0 is required: a negative open count is never a legal state.